

Authority Firewall: Latent Instruction-Hierarchy Monitoring for Safe Personalized Agents

Aleksandar L. Slavov*

Baba Tonka High School of Mathematics
Bulgaria

Cynthia Lambov*

Sofia High School of Mathematics High School
Bulgaria

Abstract

Personal AI agents increasingly rely on persistent user models, long-term memory, retrieval, and tool access to adapt to individual users. This personalization introduces a fundamental control problem: inputs arrive through channels with differing levels of authority. System-level instructions should override persistent user personalization, while retrieved context should inform the agent without directly controlling its behavior. We investigate whether this channel-based authority hierarchy is represented as a measurable latent signal in large language models, and whether such a signal can serve as a lightweight safety primitive for user-adaptive agents. Across multiple models, we find that authority is linearly decodable from hidden states as a recoverable geometric direction. We introduce *Authority Margin*, a representation-level diagnostic that quantifies the separability of authority levels across layers and architectures. Results further suggest that hierarchical control signals are explicitly organized within representation space. Building on these findings, we propose *Authority Firewall*, a lightweight monitoring mechanism that detects authority-shift signatures in latent activations and routes, escalates, or blocks suspicious inputs before they influence a personalized agent. On MoBChallenge, a controlled benchmark of benign and adversarial prompts, Authority Firewall achieves high recall while reducing latency by approximately 22× relative to a DeBERTa-based detector. When integrated into a cascaded security pipeline, it improves F1 from 0.941 to 0.952, reduces latency by 52.09%, and lowers expected filtering cost by 67.94%. These results suggest that safe personal intelligence systems require monitoring not only what an agent knows about a user, but also which input channels are permitted to influence the agent’s behavior.

CCS Concepts

• **Security and privacy** → **Software and application security**; • **Computing methodologies** → **Artificial intelligence**; *Natural language processing*; • **Human-centered computing** → *Human computer interaction (HCI)*.

Keywords

personalized agents, user-adaptive AI, instruction hierarchy, prompt injection, authority confusion, latent monitoring, LLM safety, controllability

*Both authors contributed equally to this research.

1 Introduction

Personalized agents rely heavily on external context to generate accurate and up-to-date responses. While this capability is central to their usefulness, it also introduces significant security risks. Retrieved emails, webpages, calendar entries, or tool outputs may contain adversarial instructions designed to hijack the agent by reframing the user request, overriding safety policies, or altering operational constraints.

This creates a fundamental security challenge for user-adaptive systems. Personal agents are expected to prioritize system guidelines and user instructions (*higher-authority* inputs) over retrieved external content (*lower-authority* input). However, current models often fail to reliably distinguish between these sources, leading to *authority confusion*: lower-authority content is incorrectly treated as a higher-priority instruction. In personalized agents, such failures are particularly dangerous because they may influence not only the current response, but also future agent behavior.

Preventing lower-authority inputs from acquiring unintended control is therefore essential for safe and aligned agent behavior. Despite the importance of instruction hierarchy, it remains unclear whether large language models internally represent authority structure in a systematic and accessible manner.

In this work, we show that instruction authority is consistently linearly decodable from the hidden representations of large language models. Across multiple architectures and layers, hidden states align along a coherent low-dimensional direction reflecting the hierarchy:

system-level policy > user-authorized personalization > external context.

This finding suggests that instruction hierarchy constitutes an internal geometric feature of model representations.

Building on this structure, we introduce *Authority Margin*, a representation-level diagnostic that measures the geometric separation between authority levels across layers and architectures. We further propose *Authority Firewall* (AF), a lightweight runtime monitor that detects authority-shift signatures directly from latent activations. AF can route, escalate, or block suspicious inputs before they influence agent behavior.

To evaluate this setting, we introduce MoBChallenge, a benchmark containing both benign and adversarial prompt-injection and authority-confusion scenarios. When integrated into a cascaded security pipeline, AF improves F1 from 0.941 to 0.952, while reducing end-to-end filtering latency by 52.09% and lowering expected compute cost by 67.94%.

Contributions.

- We show that instruction authority is linearly decodable from hidden representations across multiple large language model architectures and layers.
- We introduce *Authority Margin*, a diagnostic metric for quantifying geometric separation between authority levels in latent space.
- We introduce MoBChallenge, a benchmark containing benign and adversarial prompt-injection and authority-confusion scenarios.
- We develop *Authority Firewall*, a lightweight latent-space monitor that improves detection performance while substantially reducing inference latency and filtering cost in cascaded security pipelines.

2 Personal-Agent Threat Model

Personal agents differ from generic chatbots because they operate over persistent, user-specific state. A deployed personal agent may maintain preferences, routines, task histories, private documents, calendar context, emails, tool permissions, and long-term objectives. While these data sources enable deeper personalization and more capable assistance, they also substantially expand the attack surface: the agent must continuously distinguish between text that should *inform* behavior and text that is attempting to *control* behavior.

In practical deployments, agent inputs typically originate from three broad authority channels. The highest-authority channel is the *system* channel, which encodes provider policies and deployment constraints. The intermediate channel is *user-authorized personalization*, which captures persistent user-level customization such as preferences, standing instructions, and long-horizon goals. The lowest-authority channel is *external context*, which consists of retrieved or observed content including emails, documents, web-pages, memory snippets, and tool outputs.

The contextual channel is particularly susceptible to *authority-confusion attacks*, in which low-authority content attempts to override higher-authority instructions. Examples include a retrieved document instructing the agent to ignore user preferences, a web-page requesting disclosure of private memory, or a tool output attempting to elicit credentials or privileged information. Personal agents are especially vulnerable to such attacks because they routinely ingest large volumes of untrusted external content while simultaneously maintaining access to sensitive user state and high-impact capabilities. Existing guardrail mechanisms can mitigate many of these threats, but strong defenses are often computationally expensive and introduce additional latency.

Our objective is not to replace downstream safety classifiers or policy enforcement mechanisms. Instead, we propose a lightweight representation-level monitor that detects whether an input exhibits latent signatures of high-authority control despite originating from a lower-authority channel. Such a monitor is particularly valuable in personal-agent settings because it can function as an early routing layer.

3 Related Work

Personalized agents and user-adaptive systems. Recent work on personalized agents and user-adaptive systems has emphasized the integration of long-term memory, contextual retrieval, preference

modeling, and continual adaptation in order to provide more tailored and persistent assistance. These systems rely on the ability to incorporate heterogeneous user-specific information sources, including interaction history, stored memories, documents, and evolving behavioral patterns.

Prompt injection and cascaded defenses. With the deployment of LLMs in tool-using and retrieval-augmented systems, prompt injection and jailbreak attacks have become central security concerns [13]. Modern systems often use input guardrails [18] and cascaded security pipelines, where filters of increasing cost are applied sequentially and later stages are invoked only for ambiguous or high-risk inputs [1, 6, 16]. This defense-in-depth structure is especially natural for personal agents, where cheap routing decisions can reduce latency while preserving stronger checks for sensitive cases.

Representation-level monitoring. A growing body of work shows that transformer hidden states encode high-level properties in structured and often approximately linear ways. Linear probes have been used to recover linguistic and semantic attributes from frozen activations [2, 8–10]. Other work studies latent directions corresponding to meaningful model behaviors and uses projection-based diagnostics to interpret internal representations [5, 7, 11]. Our work extends this perspective to instruction authority and uses the resulting signal as a practical monitor.

Instruction hierarchy and authority. Instruction-tuned models are trained to follow prompts from multiple sources, and deployed applications commonly distinguish system, developer, and user instructions [4, 12]. Prior work has explored role confusion as a prompt-injection mechanism [17], but it remains unclear whether authority is consistently and linearly accessible from model representations. We address this gap by measuring authority geometry across models and by applying it to personal-agent safety.

4 Methodology

Note. All experiments are conducted both on a multinomial and a binary basis. Our motivation for this is described in Section 6.

4.1 Linear Probe and Authority Axis

To test whether instruction authority is linearly decodable from model representations, we train a logistic regression probe on frozen hidden states from each model, demonstrating that the observed linear separability is not specific to a single architecture. For each input prompt, let $h \in \mathbb{R}^d$ denote its extracted representation and let $y \in \{\text{context}, \text{personalization}, \text{system}\}$ denote its authority-channel label. The probe computes class logits

$$z = Wh + b, \quad (1)$$

where $W \in \mathbb{R}^{3 \times d}$ and $b \in \mathbb{R}^3$ in the multinomial setting, and $W \in \mathbb{R}^{2 \times d}$ and $b \in \mathbb{R}^2$ in the binary setting. The predicted role is given by

$$\hat{y} = \arg \max_c z_c. \quad (2)$$

However, linear decodability alone does not imply that there is a causal relation between the model and this feature during generation. To obtain a single direction associated with instruction

authority, we use the ordered structure of the labels. Since the authority hierarchy is

$$\text{system} > \text{personalization} > \text{context},$$

we define the authority axis as the pairwise discriminative direction between the highest- and lowest-authority classes:

$$\mathbf{w}_{\text{auth}} = \mathbf{W}_{\text{system}} - \mathbf{W}_{\text{context}}, \quad (3)$$

where $\mathbf{W}_{\text{system}}$ and $\mathbf{W}_{\text{context}}$ denote the corresponding rows of the learned weight matrix. This direction points toward representations that the probe associates more strongly with system-level than context-level inputs. The intermediate personalization class is then expected to occupy an intermediate region along this axis if authority is encoded as an approximately ordered latent variable.

For comparability across layers and models, we use the normalized authority direction

$$\bar{\mathbf{w}}_{\text{auth}} = \frac{\mathbf{w}_{\text{auth}}}{\|\mathbf{w}_{\text{auth}}\|_2}. \quad (4)$$

4.2 Authority Projection Score

Given a representation $\mathbf{h} \in \mathbb{R}^d$ and the normalized authority axis $\bar{\mathbf{w}}_{\text{auth}}$, we define the authority projection score as

$$s(\mathbf{h}) = \bar{\mathbf{w}}_{\text{auth}}^\top \mathbf{h}. \quad (5)$$

This scalar score gives the position of the representation along the learned authority direction. Higher values indicate that the representation is closer, according to the probe, to the system side of the authority axis, while lower values indicate proximity to the context side.

4.3 Authority Shift

To make the projection score easier to interpret, we first standardize it using the empirical mean and standard deviation of benign validation examples, and then map the standardized score to the unit interval using a sigmoid link function. This produces an authority-shift score

$$a = \text{sigmoid}\left(\frac{s(\mathbf{h}) - \mu_0}{\sigma_0}\right), \quad (6)$$

where μ_0 and σ_0 are estimated only from benign validation examples.

The affine standardization $\frac{s(\mathbf{h}) - \mu_0}{\sigma_0}$ expresses deviations from the benign distribution in units of benign standard deviation, while the sigmoid function maps this normalized deviation to a (0, 1)-valued score. Since the sigmoid is strictly monotone, thresholding a is equivalent to thresholding s directly. Thus, the transformation improves interpretability without changing the underlying ordering or decision structure. The use of benign-only calibration follows the one-class detection principle: the decision boundary is defined relative to the inlier distribution without requiring access to the attack distribution at calibration time [15]. A more detailed explanation of a is provided in Appendix A.

4.4 Authority Margin

To quantify the separation between high- and low-authority channels, we define the *Authority Margin* metric as the standardized

difference between the projected system and context distributions:

$$\text{AM} = \frac{\mu_{\text{system}} - \mu_{\text{context}}}{\sqrt{\frac{(n_{\text{system}} - 1)\sigma_{\text{system}}^2 + (n_{\text{context}} - 1)\sigma_{\text{context}}^2}{n_{\text{system}} + n_{\text{context}} - 2}}}, \quad (7)$$

where μ_{system} and μ_{context} are the mean projection scores of the system and context classes, respectively.

This quantity is analogous to Cohen’s d effect size, but is interpreted geometrically: it measures how many pooled standard deviations separate system and context inputs along the learned authority axis. A larger Authority Margin indicates stronger linear separation between high- and low-authority representations, while a smaller margin indicates weaker separation or greater overlap.

Interpretation. If instruction authority is encoded as an approximately linear latent variable, then projecting representations onto $\bar{\mathbf{w}}_{\text{auth}}$ should organize examples according to their authority level. Under this hypothesis, differences in class means reflect directional separation, while within-class variance reflects dispersion around the authority axis. Standardizing by the pooled standard deviation makes the margin invariant to rescaling of the axis and allows comparison across layers, models, or experimental settings.

4.5 Baselines

We compare against a strong classifier baseline based on DeBERTa [14], representative of current prompt-injection defenses. Such models achieve high accuracy but are computationally expensive and less interpretable.

We establish a baseline detection cascaded security pipeline (CSP) modeled after classical architectures, such as production-style solutions inspired by systems like Amazon Bedrock Guardrails [3]. The baseline CSP consists of the following stages applied in order:

- (1) **Lexical Filter** - matches known prompt injection phrases via case-insensitive substring search. Inputs matching any pattern are immediately blocked.
- (2) **PII Filter** - detects personally identifiable information including email addresses, phone numbers, and credit card numbers.
- (3) **Denied Topic Filter** - targets explicitly harmful intent categories: malware generation, credential theft, phishing, and instructions for physical harm, matched against a curated deny-list.
- (4) **Neural Classifier** - fine-tuned DeBERTa-v3-base model [14] with a blocking threshold of $\theta_{\text{DeBERTa}} = 0.80$. This stage provides high recall on adversarial inputs but is the dominant source of pipeline latency, making universal invocation costly at scale.

We insert our method as an early-stage filter and evaluate whether it reduces downstream load while maintaining detection performance, thereby improving both latency and cost-efficiency.

5 Experiments

5.1 Data

We use the openai/ih-challenge dataset, designed to study instruction-following behavior and robustness to prompt-injection attacks. The dataset contains prompts associated with different roles that form a

hierarchy of authority, including *system* and *developer*. Placeholders are also provided for *user* prompts.

For our analysis, we extract role-specific text segments and construct a balanced dataset across the three classes. To reduce superficial correlations, we remove duplicates and generate semantically equivalent paraphrases of *developer* and *system* prompts in the style of *user* requests using Qwen2.5-7B. We then assign numerical authority labels for probing.

We interpret the original role annotations as proxies for the authority hierarchy present in personal-agent systems. This mapping is justified because both settings share the same underlying structural property: instructions originate from channels with different levels of control authority, and the model must preserve the intended precedence ordering between them. In the original dataset, *system* prompts represent the highest-priority behavioral constraints, *developer* prompts represent intermediate task-level guidance, and *user* prompts represent lower-priority natural-language inputs. Personal-agent architectures exhibit an analogous hierarchy in which provider policies dominate persistent personalization, which in turn dominates transient contextual content such as retrieved documents, webpages, emails, and tool outputs.

Under this interpretation, system-level prompts correspond to provider-enforced policy and deployment constraints, developer-level prompts correspond to persistent customization or standing instructions (*personalization*), and user-level prompts correspond to low-authority contextual inputs (Table 1). Although the dataset does not contain realistic agent artifacts such as emails or retrieved webpages, our objective is not to model the surface form of those inputs. Instead, we study whether authority level itself is encoded in model representations independently of domain-specific content. The abstraction is therefore appropriate because the core research question concerns hierarchical control structure rather than the specific modality or source of the text.

Table 1: Mapping from source role labels to the personal-agent authority hierarchy.

Source label	Agent channel	Control role
System	System	Provider policy
Developer	Personalization	Standing user/app instructions
User	Context	Low-authority natural-language input

MoBChallenge. To evaluate monitoring, we introduce *MoBChallenge* (Malicious or Benign Challenge), a compact controlled benchmark of benign and adversarial prompts. It includes explicit authority manipulation, instruction override, role-play, obfuscation, and jailbreak-style attacks. The benchmark contains 630 prompts, of which 44% are benign and 56% are adversarial. It is designed as a diagnostic benchmark for authority-confusion detection rather than a complete real-world security benchmark.

Although *MoBChallenge* is not restricted to personal-agent scenarios, its attack templates model the core failure that personalized agents must handle: lower-authority text attempting to behave like a higher-authority instruction. In deployment, such text may originate from retrieved documents, emails, webpages, memory

entries, or tool outputs. We therefore evaluate Authority Firewall as an early-stage monitor in a cascaded pipeline rather than as a standalone guarantee.

5.2 Experimental Setup

We evaluate six transformer models:

Meta-Llama-3.2-3B-Instruct, Phi-3-mini-4k-instruct, Qwen2.5-3B-Instruct, DeepSeek-R1-Distill-Qwen-1.5B, and quantized versions of Mistral-7B-Instruct-v.3 and Gemma-2-2b-it. For each model, we extract hidden states for input prompts and use the final-token representation as a compact summary of the sequence. To study how authority signals evolve, we collect activations from all layers. Additional details are provided in Appendix B.

6 Results

6.1 Classification performance

As shown in Table 2, the linear probe achieves strong performance in distinguishing the three authority-channel classes: *context*, *personalization*, and *system*. We further analyze the binary control boundary by aggregating the *system* and *personalization* classes into a single control-authority category. In the evaluated setting, this produces perfect separation from the low-authority *context* class, suggesting that the models encode authority most strongly as a control-versus-context distinction rather than as a fully separated three-level taxonomy.

Table 2: Linear probe performance. M: multinomial; B: binary.

Model	M-Acc	M-F1	M-AUC	B-Acc	B-F1	B-AUC
Llama-3.2	0.800	0.800	0.917	1.00	1.00	1.00
Gemma-2B	0.744	0.742	0.891	1.00	1.00	1.00
Phi-3	0.783	0.783	0.912	1.00	1.00	1.00
Qwen2.5	0.733	0.732	0.901	1.00	1.00	1.00
Mistral	0.783	0.783	0.912	1.00	1.00	1.00
DeepSeek	0.728	0.728	0.874	1.00	1.00	1.00

This matches the practical structure of deployed agents, where system and personalization instructions often function as higher-authority constraints relative to context. We therefore aggregate *system* and *personalization* into a single high-authority class and use the binary setting for monitoring. From here on, all experiments shown use this binary setting; multinomial results are shown in Appendix D.

6.2 Projection analysis

To analyze the learned authority axis, we project representations onto it and examine projection distributions across roles. Figure 1 shows that context prompts consistently occupy lower projection regions than personalization and system prompts, supporting the interpretation of authority as an ordered latent direction.

6.3 Geometric structure of representations

We apply Principal Component Analysis (PCA). This allows us to assess whether authority corresponds to a primary axis of variation

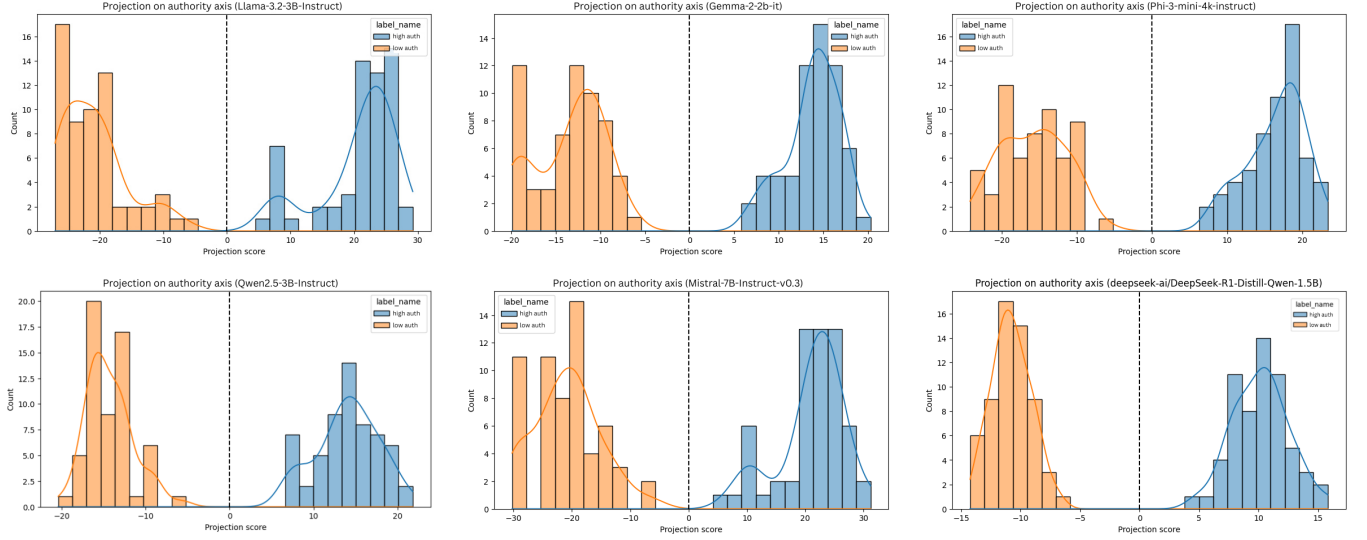


Figure 1: Projection distributions along the learned authority axis for each evaluated model.

and to visualize the relative positioning of different roles in a low-dimensional representation space.

6.4 Layer-wise analysis

We analyze how authority encoding evolves across layers by computing Authority Margin at different depths. As shown in Table 3, most models obtain AUC-AM values close to 2.1, indicating sustained authority separation across depth.

Table 3: Layer-wise Authority Margin summary across models.

Model	AUC	AM _{max}	Peak	Range	d _{peak}
DeepSeek-R1	2.067	2.110	16	0–28	0.571
Qwen2.5-3B	2.065	2.131	36	0–36	1.000
Phi-3-mini	2.102	2.152	32	0–32	1.000
Gemma-2B	2.113	2.151	13	0–26	0.500
Mistral-7B	2.103	2.161	17	0–32	0.531
Llama-3.2	2.091	2.155	27	0–28	0.964

AUC denotes normalized area under the layer-wise Authority Margin curve.

Peak locations vary substantially across architectures: Qwen2.5-3B-Instruct and Phi-3-mini-4k-instruct peak at the final layer, Gemma-2-2B-it and Mistral-7B-Instruct-v0.3 peak near the middle layers, and Llama-3.2-3B-Instruct peaks late in the network. This suggests that authority signals are not isolated to a single layer, but remain accessible across substantial portions of the representation stack.

7 Authority Firewall

Prompt-injection attacks can be detected by inconsistencies between an input’s declared role and its position along the learned authority axis. Inputs from lower-authority sources that project into high-authority regions can be flagged as anomalous. Unlike

keyword-based defenses, this approach operates on latent semantic structure and is therefore robust to paraphrasing and obfuscation.

We propose *Authority Firewall* (AF), a lightweight detector that operates directly on hidden-state representations at inference time. Its marginal cost is solely defined by a single matrix-vector product and a logistic regression forward pass, adding negligible latency.

7.1 Design Goal

The firewall is designed for settings where an agent receives both user instructions and contextual inputs from memory, retrieval, documents, webpages, or tool outputs. In such systems, a lower-authority input may be useful as evidence while still being unsafe as a behavioral command. Authority Firewall therefore does not attempt to classify all harmful content. Instead, it detects authority-confusion signatures: cases where an input originating from a lower-authority channel is represented as unusually close to high-authority instructions.

The firewall is intentionally used as a routing layer rather than as a standalone safety guarantee. Clearly benign inputs are allowed, clearly suspicious inputs are blocked, and uncertain inputs are escalated to a stronger downstream detector. This matches the deployment needs of personal agents: latency should remain low for ordinary interaction, but inputs that may affect user memory, private context, or tool behavior should receive stronger inspection.

7.2 Signals

For each input, Authority Firewall uses three scalar signals:

- (1) the authority projection score $s(h)$, which measures alignment with the learned authority direction;
- (2) the binary probe probability $\hat{p}_{\text{attack}}$, which estimates whether the representation resembles a high-authority instruction;
- (3) the calibrated authority shift a , which measures deviation from benign validation traffic after standardization.

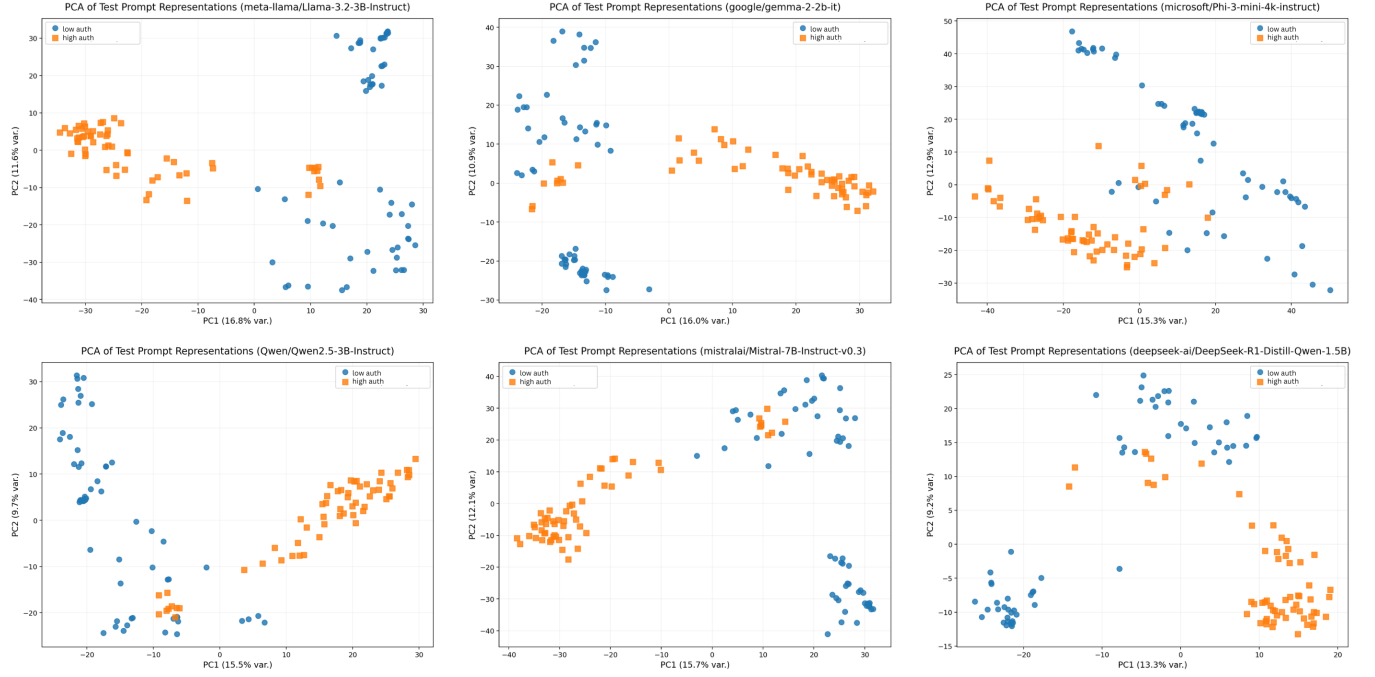


Figure 2: PCA visualization for each model.

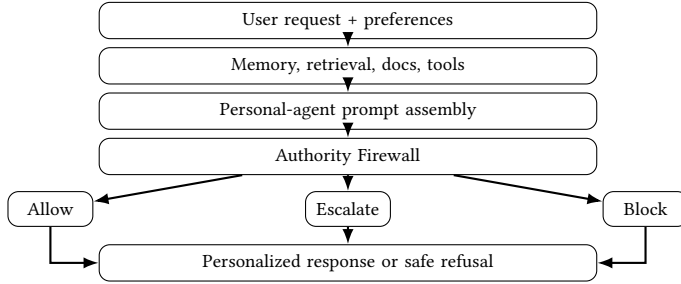


Figure 3: Authority Firewall as a representation-level safety layer for personalized agents. Contextual sources such as retrieved documents or tool outputs may be useful evidence, but should not override persistent user personalization or provider-level system policy.

These signals capture different views of the same phenomenon: geometric position, classifier confidence, and calibrated anomaly strength. Using all three makes the routing rule more conservative than a single-threshold detector.

7.3 Routing Policy

Given signals $s(h)$, $\hat{p}_{\text{attack}}$, and a , upper thresholds $\tau_{\text{high}} = (\theta_s, \theta_p, \theta_a)$, and lower thresholds $\tau_{\text{low}} = (\theta_s^-, \theta_p^-, \theta_a^-)$, the firewall applies the routing rule shown in Figure 4. Inputs are blocked only when all high thresholds are exceeded. Inputs are escalated when any lower threshold is exceeded. Otherwise, inputs are allowed.

The asymmetric rule reflects the different costs of errors. Blocking requires stronger evidence because false positives can interrupt

$$\rho = \begin{cases} \text{block} & s(h) > \theta_s \wedge \hat{p}_{\text{attack}} > \theta_p \wedge a > \theta_a \\ \text{next_stage} & s(h) > \theta_s^- \vee \hat{p}_{\text{attack}} > \theta_p^- \vee a > \theta_a^- \\ \text{allow} & \text{otherwise.} \end{cases}$$

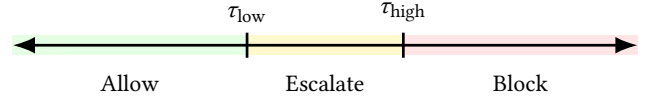


Figure 4: Authority Firewall routing rule and threshold interpretation. The equation gives the logical routing policy; the line below shows the corresponding allow-escalate-block regions.

benign user interaction. Escalation is deliberately easier because it only invokes a stronger detector; it does not irreversibly reject the prompt.

7.4 Standalone Detection

We first evaluate Authority Firewall as an independent detector on MoBChallenge. In this setting, the firewall is compared against a DeBERTa-based prompt-injection detector. As shown in Table 4, Authority Firewall reduces latency by approximately 22× relative to DeBERTa. It has lower precision but higher recall, which is appropriate for an early-stage monitor whose primary role is to catch suspicious inputs before they reach memory, tools, or user-adaptive behavior.

Table 4: Standalone comparison between Authority Firewall and DeBERTa on MoBChallenge. Latency is reported in milliseconds.

Predictor	Latency	Precision	Recall	F1	ASR
Authority Firewall	0.331	0.822	0.948	0.881	0.051
DeBERTa	7.335	0.993	0.874	0.930	0.126

7.5 Cascaded Security Pipeline Integration

We next insert Authority Firewall between the rule-based filters and the neural classifier in the baseline cascaded pipeline. This allows the system to avoid invoking the expensive classifier for inputs that are clearly benign or clearly suspicious under the authority monitor, while still escalating ambiguous inputs.

When inserted into the cascaded pipeline, Authority Firewall improves F1 from 0.941 to 0.952 and reduces average latency by 52.09%, as shown in Table 5. The expected filtering cost is reduced by 67.94%, as shown in Table 6. These results support the use of representation-level monitoring as a cheap routing mechanism for personal-agent safety.

Table 5: Cascaded security pipeline comparison. Latency is reported in milliseconds. AF-CSP reduces average latency by 52.09%.

Pipeline	Lat.	Prec.	Rec.	F1	ASR
Baseline	4.756	0.9937	0.8943	0.9414	0.1057
AF-CSP	2.278	0.9934	0.9143	0.9524	0.0857

Table 6: FLOP-based computational comparison between the baseline detector pipeline and the routed Authority Firewall pipeline.

Quantity	Value
Average DeBERTa sequence length	14.78 tokens
DeBERTa cost / prompt	2.519 GFLOPs
AF marginal cost / prompt	1.8452×10^{-5} GFLOPs
Baseline expected cost / prompt	2.407 GFLOPs
CSP with AF expected cost / prompt	0.772 GFLOPs
Saved cost / prompt	1.636 GFLOPs
Expected FLOP reduction	67.94%
DeBERTa / AF marginal cost ratio	136,540×

8 Discussion

Our results provide consistent evidence that instruction authority is encoded as a structured, approximately linear feature in the latent space of instruction-tuned models. Classification results, projection analyses, and layer-wise Authority Margin measurements all support the interpretation of a coherent authority direction. Under our personal-agent framing, this direction separates low-authority *context* from higher-control channels such as *personalization* and

system. This suggests that authority is not merely a prompt-format convention, but a geometrically accessible property of hidden-state representations.

A central empirical pattern is that the strongest separation appears between context and control. In the multinomial setting, the intermediate *personalization* class overlaps more with *system* than with *context*. In the binary setting, aggregating *system* and *personalization* into a single control-authority category yields perfect separation from *context* in the evaluated setting. This supports the view that models encode authority most robustly as a control-versus-context distinction rather than as a perfectly separated three-level taxonomy.

The projection and layer-wise results further strengthen this interpretation. Context inputs consistently receive lower authority projection scores than personalization and system inputs, indicating that the learned authority axis captures an ordered latent dimension rather than an arbitrary classifier boundary. Authority Margin remains strong across substantial portions of the transformer stack, although peak layers vary by architecture. This suggests that authority information is broadly available in representations, while the optimal monitoring layer may be model-specific.

Authority Firewall turns this geometric structure into a practical safety mechanism. Its goal is not to classify every high-control signal as malicious: a genuine system instruction should look high-authority. The security problem arises when control-like semantics appear in a low-authority channel such as retrieved documents, webpages, emails, memory snippets, or tool outputs. AF directly targets this mismatch between channel authority and latent control semantics.

The standalone results show that AF has the profile of an effective early-stage monitor. Compared with a DeBERTa-based detector, it reduces latency by approximately 22× while maintaining high recall on adversarial prompts. Its lower precision is acceptable in a routing setting because suspicious but uncertain inputs can be escalated rather than immediately blocked. This trade-off is especially appropriate for personal agents, where missed authority-confusion attacks may affect memory, tool use, private context, or future personalization.

The cascaded pipeline results show that the signal is operationally useful, not merely interpretable. Adding AF improves F1 from 0.941 to 0.952, reduces average latency by 52.09%, and reduces expected filtering cost by 67.94%. These gains come from using latent authority monitoring as an intermediate routing stage: clearly benign inputs can bypass expensive classification, clearly suspicious inputs can be blocked early, and uncertain inputs can still be escalated.

For personal intelligence, the broader implication is that safety should not be limited to detecting harmful content. A prompt can be harmless in topic but dangerous in control semantics. For example, a retrieved webpage asking the agent to overwrite user preferences may not be toxic, but it attempts to cross the boundary from context into control. Our results suggest that representation-level authority monitoring is a practical primitive for protecting that boundary in memoryful, tool-using, user-adaptive agents.

At the same time, AF should not be interpreted as a standalone security guarantee. Linear decodability does not prove causal use of the authority direction during generation, and adaptive attackers

may exploit known thresholds. The strongest deployment use case is therefore as an efficient routing mechanism inside a broader defense-in-depth pipeline.

9 Limitations and Future Work

This study has several limitations. First, linear decodability does not prove that the model causally uses the decoded authority direction during generation. Causal interventions on the authority axis would be needed to test whether modifying this direction changes model behavior. Second, role-labeled datasets may contain lexical or formatting artifacts even after paraphrasing and deduplication. Third, MoBChallenge is compact and semi-synthetic; it should be treated as a controlled diagnostic benchmark rather than a complete real-world prompt-injection benchmark. Fourth, the reported marginal cost of Authority Firewall assumes that hidden states are available from the agent model or an existing feature-extraction pass. Deployment regimes that require an additional model forward pass would have different cost profiles.

Future work should evaluate authority monitoring on larger and more realistic personal-agent benchmarks involving memory updates, retrieval-augmented generation, email and document agents, tool-use traces, and long-horizon personalization. Another important direction is adaptive evaluation: attackers may optimize against known thresholds, so Authority Firewall should be combined with randomized routing, stronger downstream classifiers, and policy-level memory safeguards. Finally, causal representation editing could test whether authority axes are only diagnostic or can also support controllable alignment interventions.

10 Conclusion

We introduced Authority Firewall, a latent instruction-hierarchy monitor for safe personalized agents. The central claim is simple: safe personal intelligence requires monitoring not only what an agent knows about a user, but also which inputs are allowed to control the agent’s behavior. Across six instruction-tuned models, we showed that instruction authority is linearly decodable from hidden states and can be summarized using Authority Margin. We then used this geometry to build a lightweight monitor that improves a cascaded prompt-injection defense, increasing F1 from 0.941 to 0.952 while reducing latency by 52.09% and expected filtering cost by 67.94%.

As personal agents become more memoryful, tool-using, and user-adaptive, the boundary between context and instruction will become a core safety problem. Our results suggest that representation-level authority monitoring is a practical primitive for preserving that boundary.

References

- [1] İpek Abasıkeleş Turgut and Edip Gümüş. 2026. CASCADE: A Cascaded Hybrid Defense Architecture for Prompt Injection Detection in MCP-Based Systems. arXiv:2604.17125 [cs.CR] <https://arxiv.org/abs/2604.17125>
- [2] Guillaume Alain and Yoshua Bengio. 2016. Understanding Intermediate Layers Using Linear Classifier Probes. arXiv:1610.01644 [stat.ML] <https://arxiv.org/abs/1610.01644>
- [3] Amazon Web Services. 2024. Guardrails for Amazon Bedrock. Online documentation. <https://aws.amazon.com/bedrock/guardrails/> Accessed: 2026-05-05.
- [4] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn

- Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndots, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. 2022. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073 [cs.CL] <https://arxiv.org/abs/2212.08073>
- [5] Yonatan Belinkov and James Glass. 2019. Analysis Methods in Neural Language Processing: A Survey. *Transactions of the Association for Computational Linguistics* 7 (2019), 49–72. <https://arxiv.org/abs/1812.08951>
- [6] Hoagy Cunningham, Jerry Wei, Zihan Wang, Andrew Persic, Alwin Peng, Jordan Abderrachid, Raj Agarwal, Bobby Chen, Austin Cohen, Andy Dau, Alek Dimitriev, Rob Gilson, Logan Howard, Yijin Hua, Jared Kaplan, Jan Leike, Mu Lin, Christopher Liu, Vladimir Mikulik, Rohit Mittapalli, Clare O’Hara, Jin Pan, Nikhil Saxena, Alex Silverstein, Yue Song, Xunjie Yu, Giulio Zhou, Ethan Perez, and Mrinank Sharma. 2026. Constitutional Classifiers+: Efficient Production-Grade Defenses against Universal Jailbreaks. arXiv:2601.04603 [cs.CR] <https://arxiv.org/abs/2601.04603>
- [7] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndots, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. 2022. A Toy Model of Superposition. *Transformer Circuits Thread*. https://transformer-circuits.pub/2022/toy_model/index.html
- [8] Kavin Ethayarajh. 2019. How Contextual Are Contextualized Word Representations?. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. 55–65. <https://arxiv.org/abs/1909.00512>
- [9] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer Feed-Forward Layers Are Key-Value Memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 5484–5495. <https://arxiv.org/abs/2012.14913>
- [10] Evan Hernandez, Sarah Schwettmann, David Bau, Teona Bagashvili, Antonio Torralba, and Jacob Andreas. 2021. Linear Representations of Model Behavior. arXiv:2106.09685 [cs.LG] <https://arxiv.org/abs/2106.09685>
- [11] Chris Olah, Arvind Satyanarayan, Ian Johnson, Shan Carter, Ludwig Schubert, Katherine Ye, and Alexander Mordvintsev. 2018. The Building Blocks of Interpretability. Distill. <https://distill.pub/2018/building-blocks/>
- [12] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*, Vol. 35. 27730–27744. <https://arxiv.org/abs/2203.02155>
- [13] Chetan Pathade. 2025. Red Teaming the Mind of the Machine: A Systematic Evaluation of Prompt Injection and Jailbreak Vulnerabilities in LLMs. arXiv:2505.04806 [cs.CR] <https://arxiv.org/abs/2505.04806>
- [14] ProtectAI. 2024. Fine-Tuned DeBERTa-v3-base for Prompt Injection Detection. Hugging Face model card. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2> Accessed: 2026-04-10.
- [15] David M. J. Tax and Robert P. W. Duin. 2004. Support Vector Data Description. *Machine Learning* 54, 1 (2004), 45–66. doi:10.1023/B:MACH.0000008084.60811.49
- [16] Scott Thornton. 2026. TRYLOCK: Defense-in-Depth Against LLM Jailbreaks via Layered Preference and Representation Engineering. arXiv:2601.03300 [cs.CR] <https://arxiv.org/abs/2601.03300>
- [17] Charles Ye, Jasmine Cui, and Dylan Hadfield-Menell. 2026. Prompt Injection as Role Confusion. arXiv:2603.12277 [cs.CR] <https://arxiv.org/abs/2603.12277>
- [18] Giulio Zizzo, Giandomenico Cornacchia, Kieran Fraser, Muhammad Zaid Hameed, Amrisha Rawat, Beat Buesser, Mark Purcell, Pin-Yu Chen, Prasanna Sattigeri, and Kush Varshney. 2025. Adversarial Prompt Evaluation: Systematic Benchmarking of Guardrails Against Prompt Input Attacks on LLMs. arXiv:2502.15427 [cs.CR] <https://arxiv.org/abs/2502.15427>

A Authority Shift

The authority shift is a reparameterization of the raw score $s(h)$ that improves interpretability and operational robustness without altering the underlying decision structure. The transformation standardizes s using moments estimated from benign data and then

applies a monotone link function,

$$a = \sigma\left(\frac{s(h) - \mu_0}{\sigma_0}\right),$$

which maps the standardized score to a bounded interval. Because the sigmoid is strictly monotone, thresholding a is equivalent to thresholding $s(h)$ under an appropriate change of variables. No additional discriminative information is introduced in an order-theoretic sense.

Its primary effect is geometric: the normalization centers benign traffic at zero and expresses deviations in units of benign variability, while the sigmoid rescales the resulting values into a common range. This is useful in routing because a can be compared with other bounded signals. The approach aligns with one-class classification and anomaly detection, where detection is based on deviation from an estimated inlier distribution rather than explicit modeling of outliers [15]. The sigmoid output should not be interpreted as a calibrated probability unless explicitly trained for that purpose.

B Experimental Setup

Our experimental setup used for conducting all the experiments is described in 7.

C Evaluation Metrics

We evaluate performance using precision, recall, F1 score, and attack success rate (ASR). ASR measures the fraction of adversarial prompts that bypass a defense mechanism.

D Multinomial Experiments

Our multinomial results for each model are shown in Figures 5 and 6.

E Layer-Wise Authority Margin Analysis

Since authority separation may depend on depth, we compute AM_ℓ at each layer $\ell \in \{0, \dots, L-1\}$ and use normalized depth

$$t_\ell = \frac{\ell}{L-1}. \quad (8)$$

We summarize the layer-wise curve by the normalized area under the Authority Margin curve:

$$\text{AUC-AM} = \int_0^1 \text{AM}(t) dt \approx \sum_{\ell=0}^{L-2} \frac{\text{AM}_\ell + \text{AM}_{\ell+1}}{2} (t_{\ell+1} - t_\ell). \quad (9)$$

AUC-AM provides a depth-normalized measure of how persistently authority is separated across the network, making it comparable across architectures with different numbers of layers.

We additionally report the strongest layer-wise separation and its relative location:

$$\text{AM}_{\max} = \max_{\ell} \text{AM}_\ell, \quad \ell^* = \arg \max_{\ell} \text{AM}_\ell, \quad d_{\text{peak}} = \frac{\ell^*}{L-1}. \quad (10)$$

While $\text{AM}_{\max}$ captures the best-separated layer, d_{peak} indicates where authority structure emerges most strongly. Early peaks may reflect prompt-format or role-token cues, whereas late peaks suggest that instruction hierarchy becomes organized after deeper contextual processing.

F Threshold Values

The thresholds used by Authority Firewall are fixed as follows:

$$\begin{aligned} \theta_s &= 4.5, & \theta_p &= 0.60, & \theta_a &= 0.80, \\ \theta_s^- &= -15.0, & \theta_p^- &= 0.55, & \theta_a^- &= 0.60. \end{aligned}$$

These parameters were selected through an iterative optimization process on validation data.

G Broader Impact and Responsible Release

This work is intended to improve the safety and efficiency of personalized agents by identifying lightweight representation-level signals for authority-confusion detection. The main positive impact is the potential reduction of computational cost in cascaded security pipelines while preserving or improving attack detection recall.

The work also has dual-use risks. A benchmark of malicious prompts and an analysis of authority-shift signatures could help attackers design prompts that evade simple detectors. To reduce this risk, benchmark examples should be released for research evaluation with labels and metadata, while avoiding automated attack-generation scripts that optimize directly against the detector. Authority Firewall should not be deployed as a standalone security mechanism; it is intended as one component of a defense-in-depth pipeline with stronger downstream classifiers and policy checks.

The method may produce false positives on benign prompts that use assertive, administrative, or instruction-like language. Such errors could disproportionately affect users whose writing style resembles high-authority directives. For this reason, AF should primarily be used as a routing or escalation mechanism rather than an irreversible blocking mechanism in high-stakes deployments.

Table 7: Technical specifications and reproducibility details.

Component	Specification
Environment	Google Colab, Python ≥ 3.10 , dependencies installed via pip.
Core libraries	PyTorch, Hugging Face Transformers, Hugging Face Datasets.
Hardware	Single NVIDIA T4 GPU in a Colab-managed environment, with automatic device mapping.
Models	meta-llama/Llama-3.2-3B-Instruct; google/gemma-2-2b-it (quantized); microsoft/Phi-3-mini-4k-instruct; Qwen/Qwen2.5-3B-Instruct; mistralai/Mistral-7B-Instruct-v0.3 (quantized); deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B.
Data	See Section 5.1 for dataset description and preprocessing.
Training and evaluation	See Section 4 for feature extraction and probe setup.
Randomness control	Fixed seed = 12 across random, numpy, and torch; deterministic CUDA settings.
Compute budget	Approximately 25-30 GPU hours total.
Reproducibility	Code provided as an executable notebook; experiments are reproducible via sequential execution.

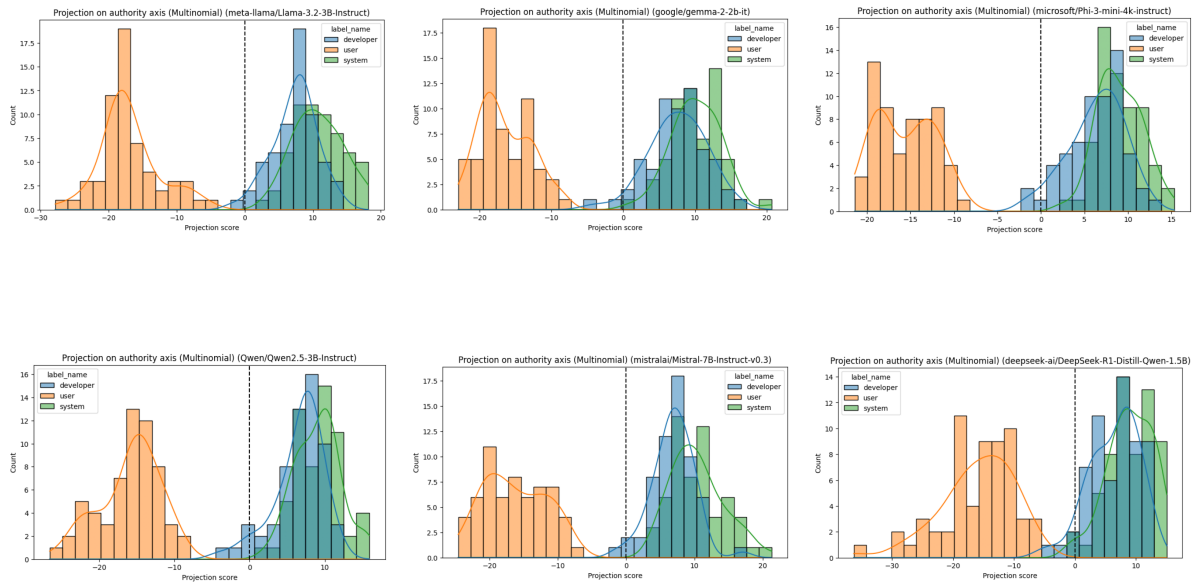


Figure 5: Projection on the multinomial authority axis for each model.

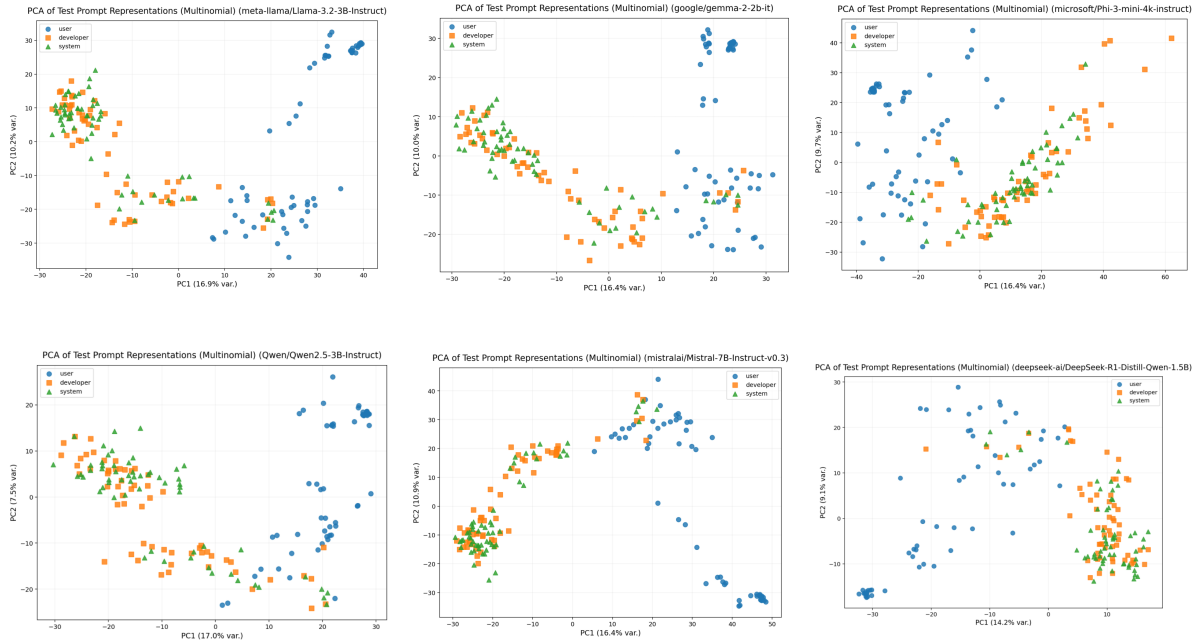


Figure 6: Multinomial PCA visualization for each model.

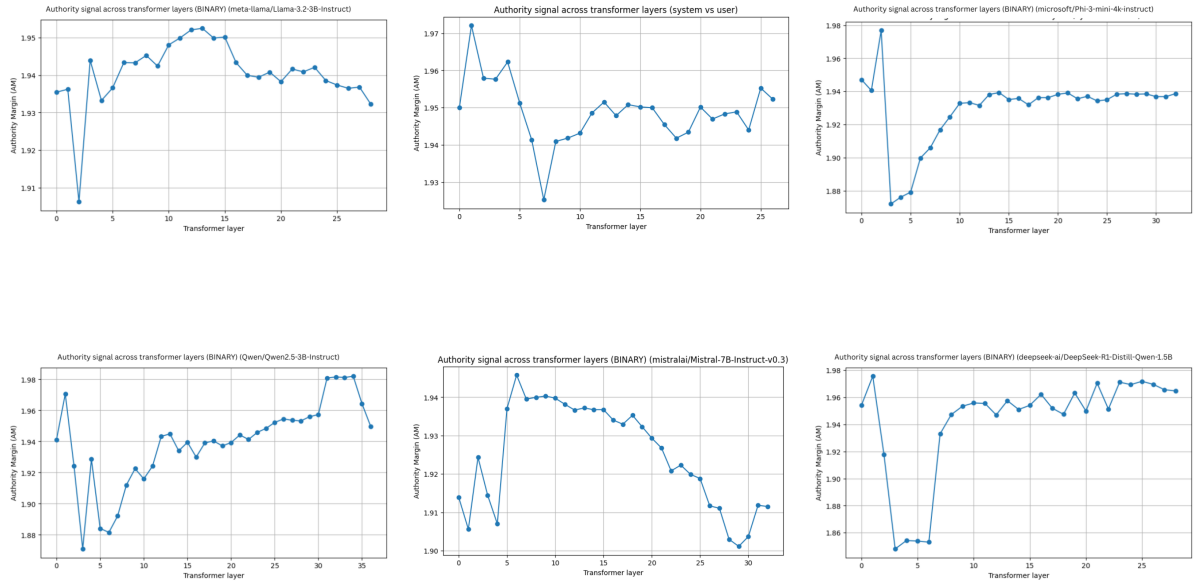


Figure 7: Layer-wise Authority Margin across evaluated models.